perplexity

what the above project do?explain it in pointwise.

The project is a **small, AI-first visual regression testing system.** It checks whether a website's appearance has changed in a way that matters, then uses local AI models to explain and report that change.

What each part does

- **Demo website** (`demo.py`)
  Runs a tiny website with two versions: a stable version and an intentionally changed version. It is only the target being tested.
- **Screenshot capture** (`capture.py`)
  Uses Playwright to open the demo website in a headless Chromium browser and take a full-page screenshot. It saves either a baseline screenshot or a current screenshot.
- **Baseline screenshot**
  This is the approved "known good" version of the page. It represents how the website should look.
- **Current screenshot**
  This is the latest version of the page. It may contain an intentional change or an accidental UI regression.
- **Pixel comparison** (`regression.py`)
  Compares the baseline and current screenshots pixel by pixel. It creates `screenshots/diff.png`, where changed areas are highlighted in red.
- **Change trigger**
  If the percentage of changed pixels is very small, the app treats it as no meaningful change. If it exceeds the threshold, it sends the screenshots to the VLM for analysis.
- **Vision model analysis** (`ai_client.py`)
  Sends three images to `qwen3-vl-2b-instruct`: baseline, current, and diff. The VLM looks at them and decides whether the change is a meaningful regression.
- **VLM output**
  The VLM returns structured information such as:
  - Whether it is a regression.
  - Severity: low, medium, high, critical, or none.
  - What visually changed.
  - Why it matters.
  - Recommended action: fix UI, investigate, or approve baseline.

- **Language model report (`ai_client.py`)**
  Sends the VLM's findings to `qwen2.5-0.5b`. The LLM converts the technical result into a short, readable QA report.
- **Markdown report (`reports/`)**
  Saves a timestamped report containing:
  - Percentage of changed pixels.
  - VLM analysis in JSON.
  - LLM-written QA summary.
  - Recommended next action.

Example flow

1. You capture the stable website as `baseline.png`.
2. You change the website's button color and text.
3. You capture the changed website as `current.png`.
4. `regression.py` finds that some pixels changed.
5. The VLM examines the images and says: "The primary button changed color and text; this may be a UI regression."
6. The LLM writes: "Regression detected. Investigate whether the CTA change was intentional before updating the baseline."
7. You get a Markdown report in `reports/`.

What the AI actually decides

- **Pixel diff:** "Some pixels changed."
- **VLM:** "The button changed from blue 'Get started' to red 'Start Now!'; this looks like a UI regression."
- **LLM:** "Here is a concise QA report and recommended action."

So the project does not merely detect that screenshots differ—it uses local AI to interpret the difference, assess its impact, and produce an actionable regression report.